Mini Project Case Study

🗂 React Project Management System

A Full-Featured Task & Team Collaboration Web Application

Team Members

Akhila Anish Das · Lokendra Joshi

Bhaumikk Keer · Jaspreet Kaur Bassi

B.Tech Computer Science & Engineering | Semester II (2025–2029)
ITM Skills University | React & Modern Web Development

📋 Abstract

This mini project presents a comprehensive **React-based Project Management System** — a modern, component-driven web application designed to help teams plan, track, and collaborate on projects effectively. The system allows users to create projects, manage tasks across multiple stages, assign responsibilities to team members, set deadlines, and monitor overall project progress in real time.

Built using React's component architecture, the application leverages concepts such as state management, props-driven communication, conditional rendering, and React Hooks to deliver a dynamic, responsive user experience. The project demonstrates how industry-standard front-end practices — component reusability, unidirectional data flow, and modular UI design — come together to solve a real-world organizational problem.

This case study documents the problem background, system design, key features, technology choices, team contributions, and future roadmap for the React Project Management System — serving as both a technical reference and an academic demonstration of modern React development competencies.

🏗 Introduction

🎯 Project Overview

The React Project Management System is a single-page application (SPA) built with React that replicates core features found in popular project management tools. It provides a centralized dashboard where teams can visualize project status, manage task lifecycles from creation to completion, assign work to individuals, and track progress — all within an intuitive, browser-based interface. The application is entirely front-end driven, showcasing the power of React's declarative UI model and state-driven rendering.

👥 Team Members

#	Name	Role	Programme
1	**Akhila Anish Das**	UI Design & Component Architecture	Larry Page | B.Tech CSE
2	**Lokendra Joshi**	State Management & Logic	Larry Page | B.Tech CSE
3	**Bhaumikk Keer**	Task Module & Kanban Board	Larry Page | B.Tech CSE
4	**Jaspreet Kaur Bassi**	Dashboard & Progress Tracking	Larry Page | B.Tech CSE

🧩 Core Modules

🗂️ Project Module

- ✓ Create and name new projects
- ✓ Set project description and deadline
- ✓ View all projects on a dashboard
- ✓ Delete or archive completed projects
- ✓ Project-level progress indicator
- ✓ Color-coded project cards

✅ Task Module

- ✓ Create tasks within each project
- ✓ Assign tasks to team members
- ✓ Set priority levels (Low / Medium / High)
- ✓ Add due dates per task
- ✓ Mark tasks as complete
- ✓ Edit and delete tasks

📊 Kanban Board

- ✓ Visual columns: To Do, In Progress, Done
- ✓ Drag-and-drop task movement
- ✓ Real-time column task counts
- ✓ Filter tasks by assignee
- ✓ Filter tasks by priority
- ✓ Responsive layout for all screens

📈 Progress Dashboard

- ✓ Overall project completion percentage
- ✓ Task status summary charts
- ✓ Member-wise task distribution
- ✓ Upcoming deadlines panel

- ✓ Overdue task alerts
- ✓ Quick project-switching sidebar

🚧 Problem Statement

🔴 The Problem

Student teams and small organizations frequently struggle with coordinating work across multiple members. Without a dedicated system, tasks are tracked in spreadsheets, communication happens in fragmented chats, deadlines are missed, and no single person has a clear view of what is done, in progress, or pending. This results in duplicated effort, accountability gaps, and project delays. The absence of a lightweight, browser-based management tool makes it especially difficult for small teams who cannot afford enterprise software.

✅ The Solution — React Project Management System

Our application provides a centralized, real-time collaboration environment. By using React's component model and state management, all team members interact with a shared project structure where changes to tasks, assignments, and statuses are immediately reflected across the interface. The Kanban board offers a visual workflow while the dashboard provides analytics — giving both team members and project leads exactly the visibility they need to stay on track.

🎮 User Flow

1. User lands on the main Dashboard after opening the app.
2. User creates a new Project with a name, description, and deadline.
3. Tasks are created within the project and assigned to team members with priorities.
4. Tasks appear on the Kanban Board under the "To Do" column automatically.
5. As work progresses, tasks move across columns: To Do → In Progress → Done.
6. The Dashboard updates in real time, reflecting completion percentage and upcoming deadlines.
7. Overdue tasks are flagged with visual alerts to prompt action from the responsible member.

🌍 Real-World Relevance

Project management systems are at the core of every software company's operations. Tools like Jira, Trello, Asana, and Monday.com are built on the same foundational concepts this project demonstrates — task lifecycle management, team collaboration, and progress visualization. Building this system in React gives the team deep, practical experience with the technologies and design patterns used daily in the software industry.

⚙ Technology Stack & Methodology

🔧 Technology Stack

React 18
Component-based UI, Hooks, SPA architecture

CSS3 / Tailwind
Responsive styling, utility classes, gradient design

React Hooks
useState, useEffect, useContext for state & side effects

Context API
Global state sharing across all components

React Router
Client-side routing between Dashboard, Board, Projects

LocalStorage
Persistent browser-side data storage

🏗 System Architecture

Component Hierarchy

App (Root)
  ├── Navbar — navigation links and branding

```
├── Dashboard — project overview cards and summary statistics
│   └── ProjectCard — individual project with progress bar
├── ProjectView — expanded view of a single project
│   ├── TaskForm — create / edit task form
│   └── KanbanBoard — drag-and-drop task columns
│       └── TaskCard — individual task display
└── ProgressPanel — charts and deadline alerts
```

🔄 Data Flow Strategy

The application follows React's unidirectional data flow model. All project and task data lives in a global Context store, which is passed down through the component tree via props and context consumers. User interactions — such as creating a task or moving it across the Kanban board — trigger state updater functions, causing only the affected components to re-render. This approach ensures performance, predictability, and ease of debugging, following the same architecture used in large-scale React applications.

⚡ Key React Concepts Applied

useState — manages form inputs, task status toggles, and UI panel visibility.

useEffect — synchronizes state with LocalStorage on every update, loads saved data on mount, and triggers deadline-check logic.

useContext — allows any nested component to read or update the global project/task store without prop drilling.

Conditional Rendering — displays the TaskForm, error messages, and empty-state placeholders based on current state values.

List Rendering with .map() — renders dynamic lists of project cards, task cards, and Kanban columns from the state array.

💻 Features & Implementation

🗂 Project Management

- ✓ CRUD operations for all projects
- ✓ Project name, description, due date
- ✓ Per-project task counter display

- ✓ Progress bar auto-calculated from tasks
- ✓ Status labels: Active / Completed / Overdue
- ✓ Grouped project listing on dashboard

Kanban Board

- ✓ Three-column workflow layout
- ✓ Drag and drop via HTML5 API
- ✓ Real-time column count update
- ✓ Priority badge per task card
- ✓ Assignee avatar display
- ✓ Due date warning indicator

Member Assignment

- ✓ Assign one or multiple members
- ✓ Member filter on Kanban board
- ✓ Task count per member on dashboard
- ✓ Avatar initials auto-generated
- ✓ Team member roster management
- ✓ Workload visibility per member

Notifications & Alerts

- ✓ Overdue task visual flag
- ✓ Deadline approaching warning
- ✓ Success toast on task creation
- ✓ Confirm dialog before deletion
- ✓ Empty state placeholder messages
- ✓ Form validation error messages

Feature Evaluation Checklist

React Implementation

- ✓ Functional components throughout

- ✓ useState for local state
- ✓ useEffect for data persistence
- ✓ useContext for global store
- ✓ Props for parent-child data flow
- ✓ Controlled form components

🧩 Component Design

- ✓ Single-responsibility components
- ✓ Reusable Button component
- ✓ Reusable Modal component
- ✓ Reusable Badge component
- ✓ Prop-types validation
- ✓ Clean folder structure

🎨 UI / UX Design

- ✓ Responsive on mobile & desktop
- ✓ Vivid color scheme throughout
- ✓ Intuitive navigation sidebar
- ✓ Progress bars for all projects
- ✓ Color-coded priority badges
- ✓ Smooth transition animations

💾 Data & Validation

- ✓ LocalStorage persistence
- ✓ Data loads on app startup
- ✓ Required field enforcement
- ✓ Date validation for deadlines
- ✓ Unique task ID generation
- ✓ Graceful empty-state handling

👥 Team Contributions

Akhila Anish Das

UI Design & Component Architecture

Led the overall visual design of the application including color scheme, layout, and responsiveness. Designed and built the reusable component library — Button, Badge, Modal, Card, and Avatar. Ensured consistent styling across all pages and established the project folder structure. Coordinated UI/UX decisions and managed the final integration of all team modules.

Lokendra Joshi

State Management & Logic

Built the global Context API store that manages all project and task data. Implemented the useReducer pattern for complex state transitions, wrote all data manipulation logic (create, update, delete, filter), and handled LocalStorage synchronization via useEffect. Ensured that state changes propagate correctly to all subscribed components without unnecessary re-renders.

Bhaumikk Keer

Task Module & Kanban Board

Developed the complete Task creation form with validation, priority selection, member assignment, and due date picker. Built the Kanban Board component with three-column layout and HTML5 drag-and-drop functionality. Implemented task card rendering, priority badge logic, overdue flag calculation, and real-time column count updates as tasks move between stages.

Jaspreet Kaur Bassi

Dashboard & Progress Tracking

Designed and implemented the main Dashboard view showing all projects as summary cards with live progress bars. Built the Progress Panel with task status breakdown, member-wise task distribution, and upcoming deadline listings. Implemented overdue alert logic and integrated React Router for seamless navigation between the Dashboard, Project View, and Kanban Board.

📈 Results & Conclusion

✨ Project Achievements & Outcomes

The React Project Management System was successfully built and demonstrates a fully functional, component-driven web application that addresses real-world team coordination challenges. The application provides an end-to-end project lifecycle management experience — from project creation through task assignment, Kanban-based workflow tracking, and progress analytics — entirely within a React SPA architecture.

The project showcases the team's ability to translate industry-standard tools and patterns into a working application, applying core React concepts in a meaningful, integrated context. Each team member's contribution was well-scoped and the modular component design allowed parallel development without conflicts.

Key Accomplishments:

- Fully functional React SPA with multi-page navigation via React Router
- Global state management using Context API and useReducer across all modules
- Interactive Kanban Board with drag-and-drop task movement between columns
- Dynamic progress tracking with real-time percentage calculations
- Persistent data storage ensuring no loss of project data on page reload
- Responsive, vibrant UI that works seamlessly on desktop and mobile
- Clean component hierarchy with reusable, well-documented components
- Effective team collaboration with clearly defined individual responsibilities

🚀 Future Work

🔭 Planned Improvements & Extensions

1. 🔐 User Authentication

Objective: Add secure login and registration so each team member has a personal workspace.
Plan: Integrate Firebase Authentication or a Node.js backend with JWT tokens. Each user sees only the projects they are assigned to, with role-based permissions (Admin, Member, Viewer) controlling available actions.

2. ☁️ Cloud Database Integration

Objective: Move from LocalStorage to a real-time cloud database for cross-device collaboration.
Plan: Integrate Firebase Firestore or a REST API backend (Node.js + MongoDB). Real-time listeners via WebSockets or Firestore snapshots will push updates to all connected clients instantly.

3. 💬 In-App Comments & Activity Feed

Objective: Allow team members to comment on tasks and see a history of all project activity.
Plan: Add a comment thread to each task card and a global activity log panel. Every action — task created, moved, completed, commented — is timestamped and shown in chronological order.

4. 📊 Advanced Analytics & Reporting

Objective: Provide deeper insights into team performance and project health.
Plan: Integrate Recharts or Chart.js to render burndown charts, velocity graphs, and member productivity reports. Export reports as PDF using the browser's print API or jsPDF library.

5. 📱 Mobile App with React Native

Objective: Extend the application to native iOS and Android platforms.
Plan: Port the core logic and Context store into a React Native project. Push notifications via Expo will replace browser-based deadline alerts, enabling truly mobile-first project management.

6. 🤖 AI-Powered Task Suggestions

Objective: Use AI to help teams break down large projects into well-scoped tasks automatically.
Plan: Integrate the Claude API or OpenAI API. When a user creates a new project and enters a description, the AI suggests a set of tasks, estimates effort, and recommends member assignments based on workload data.

🕮 References

1. React Official Documentation — Components, Hooks, and Context API:
   https://react.dev/learn
2. MDN Web Docs — HTML5 Drag and Drop API:
   https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
3. React Router Documentation — Client-Side Routing in React:
   https://reactrouter.com/en/main
4. Tailwind CSS Documentation — Utility-First CSS Framework:
   https://tailwindcss.com/docs
5. GeeksforGeeks — React Context API and State Management:
   https://www.geeksforgeeks.org/reactjs-context/
6. Atlassian — Kanban Methodology and Project Management Principles:
   https://www.atlassian.com/agile/kanban
7. ITM Skills University — React & Modern Web Development Course Material, Semester II, 2025
8. Fullstack React (Book) — Anthony Accomazzo et al., Newline Publishing, 2020

🗂 React Project Management System

Akhila Anish Das · Lokendra Joshi · Bhaumikk Keer · Jaspreet Kaur Bassi

B.Tech Computer Science & Engineering | 2025–2029

ITM Skills University | React & Modern Web Development